

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Database development with PL/SQL

Reference reading

- [13 Data Concurrency and Consistency](#)
- [11 Data Concurrency and Consistency](#)



Transactions and Concurrency in Databases

Transactions and **Concurrency** are fundamental concepts in database systems, ensuring the reliability and efficiency of operations in multi-user environments.



1. Transactions

A transaction is a sequence of one or more SQL statements that a database treats as a single logical unit of work. Transactions ensure data integrity even in the presence of system failures or concurrent access.

ACID Properties

Transactions follow the **ACID** principles to maintain database integrity:

Atomicity

All operations in a transaction are completed, or none are. If any operation fails, the transaction is rolled back.

Consistency

The database moves from one valid state to another, maintaining all defined rules and constraints.

Isolation

Concurrent transactions do not interfere with each other.

Durability

Once a transaction is committed, its changes persist even if the system crashes.

Transaction Control Statements

Statement	Description	Syntax
COMMIT	Saves all changes made during the transaction permanently.	COMMIT;
ROLLBACK	Undoes changes made during the transaction, reverting to the last COMMIT or SAVEPOINT.	ROLLBACK;
SAVEPOINT	Sets a marker within a transaction to roll back to, if necessary.	SAVEPOINT savepoint_name;
SET TRANSACTION	Configures transaction properties such as isolation levels.	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

```
BEGIN
  UPDATE accounts
  SET balance = balance - 100
  WHERE account_id = 1;

  UPDATE accounts
  SET balance = balance + 100
  WHERE account_id = 2;

  COMMIT; -- Ensures both updates are saved or none are applied
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK; -- Revert changes in case of error
END;
```

Example: Transaction in PL/SQL

```
BEGIN
  SELECT * INTO v_record
  FROM employees
  WHERE employee_id = 101
  FOR UPDATE; -- Locks the row to prevent other
              transactions from modifying it

  -- Perform some operations on v_record
  COMMIT;
END;
```

Example: Concurrency in PL/SQL

Occurrence

Concurrency refers to multiple users or processes accessing the database simultaneously. Managing concurrency effectively is critical to ensure data consistency and system performance.

Problems in Concurrency

- 1. Dirty Reads:** A transaction reads uncommitted changes from another transaction.
- 2. Non-Repeatable Reads:** A transaction reads the same data twice but gets different results due to updates by another transaction.
- 3. Phantom Reads:** A transaction reads a set of rows, but new rows matching the criteria are added by another transaction.

Isolation Levels

Databases use isolation levels to balance data consistency and concurrency. The levels, from least to most strict, are:

- 1. Read Uncommitted:**
 - Allows dirty reads.
 - Minimal consistency, highest concurrency.
- 2. Read Committed:**
 - Prevents dirty reads.
 - Default level in Oracle.
- 3. Repeatable Read:**
 - Prevents dirty and non-repeatable reads.
 - Phantom reads may still occur.
- 4. Serializable:**
 - Full isolation, preventing all concurrency issues.
 - May reduce performance due to locking.

Concurrency Control Mechanisms & Best Practices for Managing Transactions and Concurrency

Concurrency Control Mechanisms

1. Locks

- **Shared Lock:** Allows multiple transactions to read but not write.
- **Exclusive Lock:** Prevents other transactions from reading or writing.

2. Multi-Version Concurrency Control (MVCC)

- Maintains multiple versions of data to allow read and write operations without blocking.
- Oracle uses undo segments to implement MVCC.

3. Optimistic Concurrency Control

- Assumes conflicts are rare and validates data consistency before committing.

4. Pessimistic Concurrency Control

- Locks resources early to prevent conflicts.

Best Practices for Managing Transactions and Concurrency

- Keep transactions short to minimize lock durations.
- Use proper isolation levels depending on the application requirements.
- Avoid holding locks for user input or long-running operations.
- Test applications for potential deadlocks and contention issues.
- Log errors and implement retry logic for transactions experiencing concurrency conflicts

Closing Remarks

As we conclude this PL/SQL course, I hope you now have a strong foundation in both the theory and practical aspects of PL/SQL programming. You have learned to:

- Work with variables, control structures, procedures, functions, and exception handling.
- Write efficient, reliable, and maintainable PL/SQL code.

Key Takeaways:

- **Practice:** Continue building projects to reinforce your learning.
- **Explore:** Dive deeper into advanced topics like performance tuning and optimization.
- **Collaborate:** Work with peers and seek help when needed to solve complex problems.

I am confident you now have the tools to succeed in your database career. Keep pushing your limits, and I look forward to seeing your future accomplishments!

Biblical Message:

"For I know the plans I have for you," declares the Lord, "plans to prosper you and not to harm you, plans to give you a hope and a future."

— **Jeremiah 29:11**

This verse reminds us that no matter the challenges ahead, we are destined for success and growth.

Inspirational Quote:

"Success is the sum of small efforts, repeated day in and day out."

— **Robert Collier**

Keep Learning!